République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée des Technologies de l'Information et de Management de Nabeul

Rapport

Mini Projet DevOps

Réalisé par :

wassim haoues

Implémentation d'un projet en se basant sur une chaîne DevOps de bout en bout

---

Encadrant Académique :
Mohamed Najeh ISSAOUI

Année universitaire : 2025–2026

Table des matières

1 Introduction **2**

2 Description du projet **2**

2.1 Positionnement du projet dans le module 2

2.2 Support applicatif utilisé 2

2.3 Architecture du support applicatif 3

3 Choix techniques **3**

4 Architecture technique devops **4**

4.1 Environnements et modes d'exécution 4

4.2 Progression générale des phases DevOps 4

4.3 Architecture DevOps 4

4.4 Composants d'infrastructure réellement présents 5

5 Pipelines DevOps réellement présents **6**

5.1 Workflows GitHub Actions 6

5.2 Logique des pipelines 7

5.2.1 Pipeline CI 7

5.2.2 Pipeline Security 8

5.2.3 Pipeline CD 8

6 Historique Git, stratégie de branches et collaboration **9**

6.1 Stratégie observable 9

6.2 Phases majeures observables 9

7 Problèmes rencontrés **10**

8 Solutions apportées **10**

9 Conclusion **10**

1. Introduction

Dans ce mini-projet, mon objectif principal n'était pas seulement de livrer une application, mais surtout de construire une chaîne DevOps complète et cohérente autour d'un cas d'usage concret. L'application **StockPro** me sert donc avant tout de support technique pour mettre en place l'exécution locale, la validation continue, la sécurité pipeline, le déploiement Kubernetes, le GitOps et l'observabilité.

Le vrai fil conducteur du travail est la progression par phases. J'ai d'abord préparé une base exécutable en local, puis j'ai ajouté la conteneurisation, la CI, les contrôles qualité et sécurité, Kubernetes sur `kind`, ArgoCD, la publication d'images, la gouvernance du flux CD, puis enfin la supervision et la stabilisation de l'ensemble. Le rapport est donc centré sur cette montée en maturité DevOps plus que sur les fonctionnalités métier elles-mêmes.

2. Description du projet

2.1. Positionnement du projet dans le module

Dans le cadre du module DevOps, j'ai utilisé une application web comme support pour démontrer une chaîne complète de livraison technique. Le périmètre applicatif existe bien, mais il reste ici secondaire par rapport à l'objectif principal : montrer comment un projet passe progressivement du code local à une chaîne industrialisée, traçable et supervisée.

Domaine	Brique DevOps mise en place
Socle applicatif	Backend Spring Boot, frontend Angular, base MySQL
Exécution locale	Docker Compose pour lancer rapidement la stack
Conteneurisation	Dockerfiles multi-stage backend et frontend
Intégration continue	GitHub Actions pour les tests, builds et contrôles
Sécurité pipeline	CodeQL, OWASP Dependency-Check, Trivy
Déploiement Kubernetes	`kind` + Kustomize avec base et overlays
GitOps	ArgoCD + overlay GitOps comme source de vérité
Registry	GitHub Container Registry pour les images versionnées
Observabilité	Prometheus, Grafana, Loki
Gouvernance CD	validation préalable des workflows requis avant publication

Cette vue montre bien que la valeur du projet, dans ce rapport, vient surtout de l'enchaînement des briques DevOps et de leur articulation.

2.2. Support applicatif utilisé

L'application support reste volontairement assez complète pour rendre la démonstration crédible : authentification, utilisateurs, entrepôts, produits, stocks, mouvements, alertes et tableau de bord. Cela me permet d'avoir un backend, un frontend, une base de données, des endpoints de santé, des métriques et plusieurs flux à valider dans la chaîne technique.

StockPro est une application web de gestion de stock multi-entrepôts. Elle permet de gérer les utilisateurs selon leur rôle, d'administrer les entrepôts et le catalogue produits, de suivre les quantités en stock, d'enregistrer les mouvements d'entrée et de sortie, et de signaler les situations de stock faible. L'application propose aussi un tableau de bord pour suivre l'activité et les indicateurs principaux, avec une interface adaptée selon le profil connecté.

Le backend expose 9 contrôleurs REST et 8 services métier. J'ai également mis en place 14 tests backend et 14 fichiers de tests frontend. Les rôles gérés dans l'application sont `ADMIN`, `GESTIONNAIRE` et `OBSERVATEUR`.

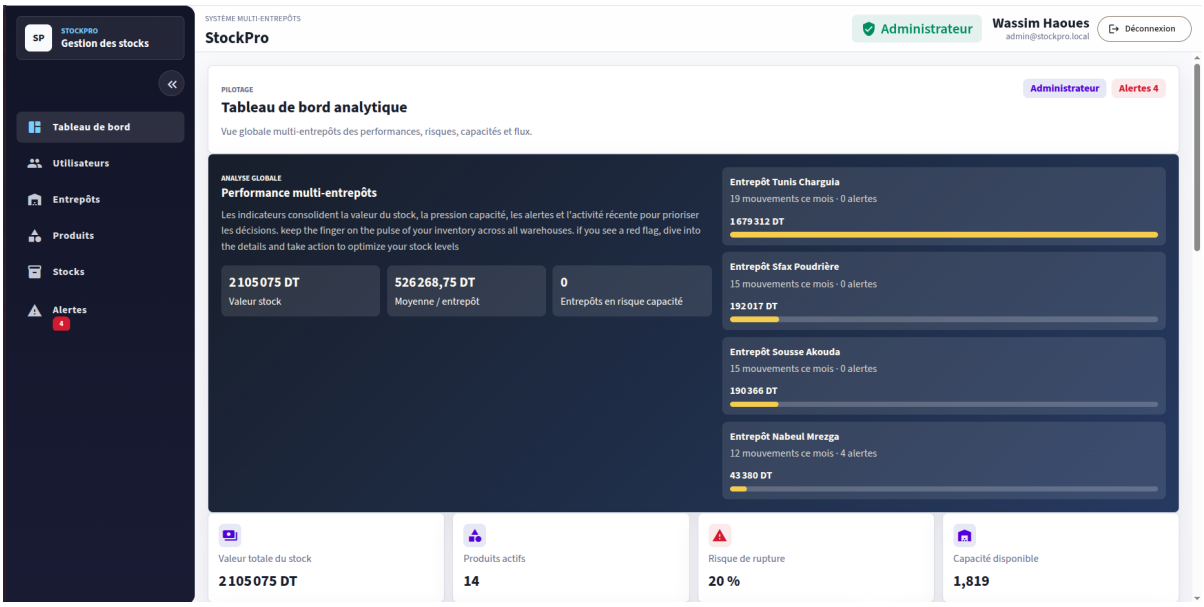


FIGURE 1 – Interface StockPro (tableau de bord)

2.3. Architecture du support applicatif

L'architecture de StockPro repose sur une séparation claire entre le frontend, le backend et la base de données. Le frontend est une application Angular servie par nginx, qui centralise l'accès utilisateur et relaie les appels API vers le backend Spring Boot. Le backend expose les services métier, applique les règles de sécurité et de gestion des rôles, puis échange avec une base MySQL pour la persistance des données.

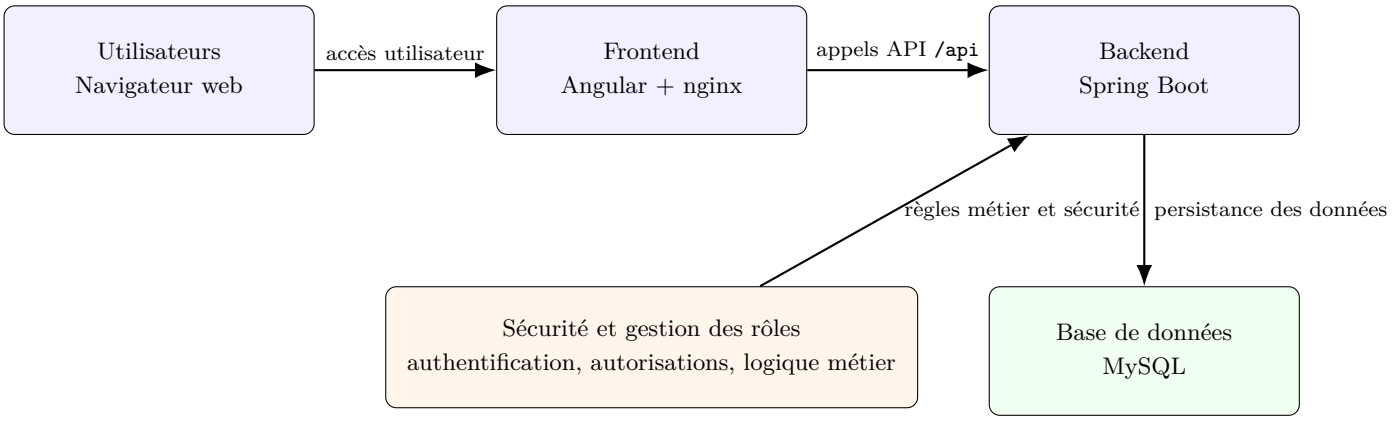


FIGURE 2 – Architecture générale de StockPro

3. Choix techniques

Dans cette partie, je mets surtout en avant les choix qui renforcent la logique DevOps du projet.

Choix	Intérêt dans la démarche DevOps
GitHub Actions	centralise la CI, les scans de sécurité, la publication d'images et la logique de validation sur le même dépôt
Docker	réduit l'écart entre développement, test local et build CI grâce à des images reproductibles
Docker Compose	permet une exécution locale rapide avant de passer à un niveau d'orchestration plus avancé
kind + Kustomize	fournit un Kubernetes local reproductible et adapté à une progression par overlays
ArgoCD	matérialise le passage d'un simple déploiement automatisé à un vrai flux GitOps piloté par Git
GHCR	permet de versionner les images avec des tags immuables liés aux commits
SonarCloud	ajoute un contrôle qualité continu au même niveau que les tests et le build

CodeQL / OWASP / Trivy	intègre la sécurité directement dans le pipeline au lieu de la traiter à part
Prometheus / Grafana	rend visible l'état du système après déploiement, et pas seulement sa réussite en CI
Loki / Promtail	complète l'observabilité avec une lecture centralisée des logs techniques
nginx reverse proxy	simplifie l'exposition du frontend et des appels API dans les différents modes d'exécution
MySQL + bootstrap SQL	garantit une base initialisée de manière reproductible dans les phases locales

4. Architecture technique devops

4.1. Environnements et modes d'exécution

J'ai organisé le projet autour de deux modes d'exécution complémentaires :

— **Docker Compose**, pour le démarrage rapide en local et la validation fonctionnelle ;
— **Kubernetes avec kind**, pour rejouer un cycle plus proche d'un environnement orchestré avec GitOps et monitoring.

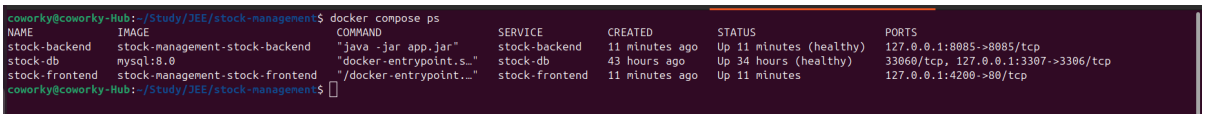
Je n'ai pas intégré Helm, Terraform, Ingress ni cible cloud explicite, parce que le but ici n'était pas de couvrir toute une plateforme de production, mais de construire une chaîne DevOps locale, complète et démontrable de bout en bout.

4.2. Progression générale des phases DevOps

La logique suivie dans le projet peut être résumée en plusieurs étapes successives :

1. mise en place d'une exécution locale simple ;
2. conteneurisation du backend et du frontend ;
3. ajout d'une CI de base ;
4. ajout des contrôles qualité et sécurité ;
5. déploiement Kubernetes local sur `kind` ;
6. mise en place d'un flux GitOps avec ArgoCD ;
7. automatisation du build et de la publication d'images ;
8. ajout du logging et de l'observabilité ;
9. durcissement du flux avec validation, gouvernance et stabilisation.

Ce découpage est important, parce qu'il montre que la chaîne DevOps n'a pas été pensée comme un bloc unique, mais comme une construction progressive où chaque phase ajoute une capacité technique concrète.



```
coworky@coworky-Hub:~/Study/JEE/stock-management$ docker compose ps
NAME            IMAGE                          COMMAND                  SERVICE          CREATED          STATUS                   PORTS
stock-backend   stock-management-stock-backend "java -jar app.jar"      stock-backend    11 minutes ago   Up 11 minutes (healthy)  127.0.0.1:8085->8085/tcp
stock-db        mysql:8.0                      "docker-entrypoint.s…"   stock-db         43 hours ago     Up 34 hours (healthy)    33060/tcp, 127.0.0.1:3307->3306/tcp
stock-frontend  stock-management-stock-frontend "/docker-entrypoint.…"  stock-frontend   11 minutes ago   Up 11 minutes            127.0.0.1:4200->80/tcp
coworky@coworky-Hub:~/Study/JEE/stock-management$
```

FIGURE 3 – Conteneurs Docker actifs avec backend, frontend et base de données

4.3. Architecture DevOps

Au-delà de l'architecture applicative, la partie la plus importante est la chaîne DevOps elle-même. Le flux que j'ai mis en place peut être résumé ainsi :

```
feature -> dev -> main
        -> CI
        -> Security
        -> build sélectif
        -> push GHCR
        -> GitOps PR
        -> validation GitOps
        -> ArgoCD
        -> kind
        -> Prometheus / Grafana
```

L'idée principale est de conserver une continuité entre le code source, l'image publiée, la mise à jour GitOps et l'état réellement déployé. Le déploiement n'est donc pas juste une exécution automatique ; il est encadré par des validations successives. Dans cette logique, le schéma suivant synthétise toute la chaîne, depuis le travail du développeur jusqu'au déploiement local et à la supervision, en distinguant clairement la phase de validation, la phase de livraison, la phase GitOps et la phase d'exploitation.

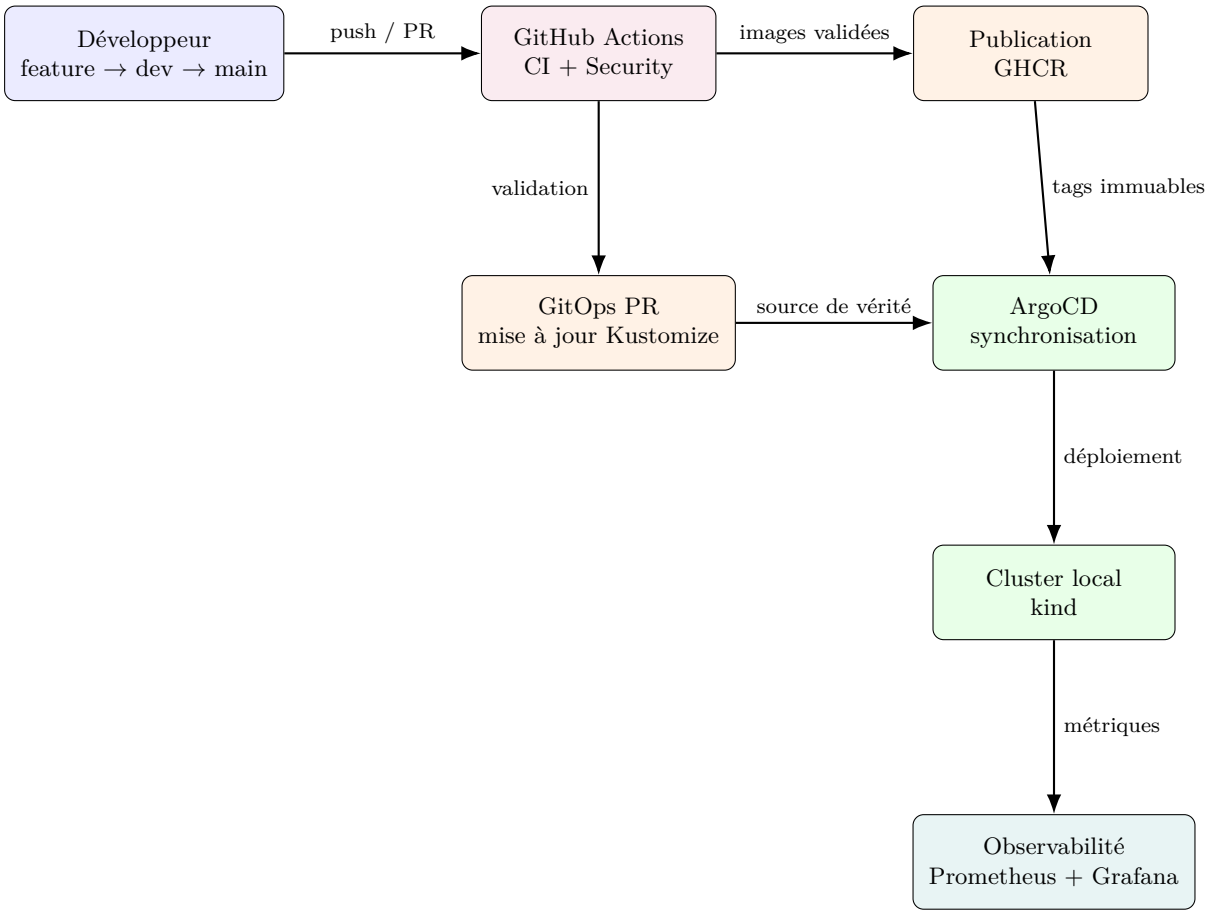


FIGURE 4 – Architecture DevOps de la chaîne de livraison

4.4. Composants d'infrastructure réellement présents

Composant	Rôle dans la chaîne DevOps
`docker-compose.yml`	lance la stack applicative locale pour la phase d'exécution initiale
`infra/docker-compose.yml`	ajoute un outil local d'administration sans alourdir la stack principale
`infra/monitoring/docker-compose.monitoring.yml`	active la supervision et la centralisation des logs en mode Docker
`k8s/base/`	contient les manifests mutualisés pour Kubernetes
`k8s/overlays/local/`	adapte les ressources à `kind` pour les tests locaux
`k8s/overlays/gitops/`	sert de source de vérité pour la synchronisation GitOps
`k8s/argocd/stockpro-app.yaml`	relie le dépôt Git à ArgoCD pour le déploiement déclaré

J'ai choisi d'intégrer Loki surtout dans la stack Docker de monitoring. Côté Kubernetes, j'ai surtout versionné Prometheus et Grafana, ce qui reste suffisant pour illustrer la supervision de la plateforme locale.

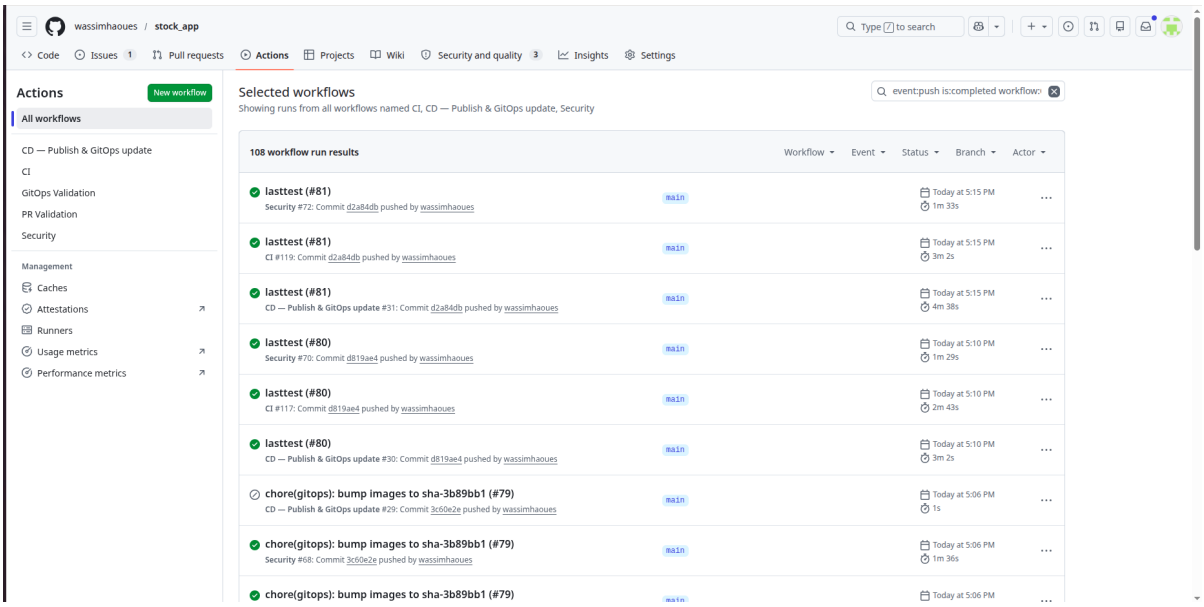


FIGURE 5 – Pods Kubernetes du namespace `stockpro` en état `Running` et services Kubernetes avec exposition NodePort

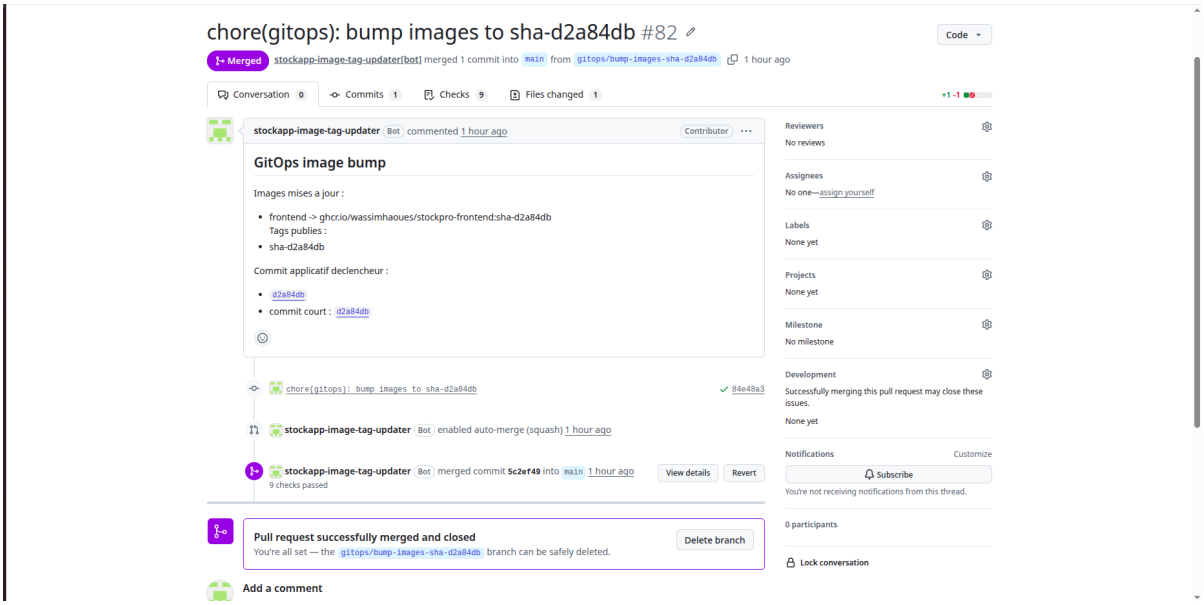


FIGURE 6 – Application ArgoCD `stockpro` en état `Synced` et `Healthy`

5. Pipelines DevOps réellement présents

5.1. Workflows GitHub Actions

Fichier	Déclenchement	Rôle
`.github/workflows/ci.yml`	push et pull request	tests backend, tests frontend, format, audit npm, build et SonarCloud
`.github/workflows/security.yml`	push, pull request, hebdomadaire	CodeQL, OWASP Dependency-Check, Trivy backend et frontend
`.github/workflows/pr-validation.yml`	pull request vers `main`	validation rapide YAML et manifests
`.github/workflows/gitops-validation.yml`	pull request vers `main`	validation légère des PR GitOps bot : yamllint, scripts Python, `kustomize build`, `kubeconform`

`.github/workflows/cd.yml`	push sur `main`	attente des validations, build sélectif, push GHCR, PR GitOps et auto-merge

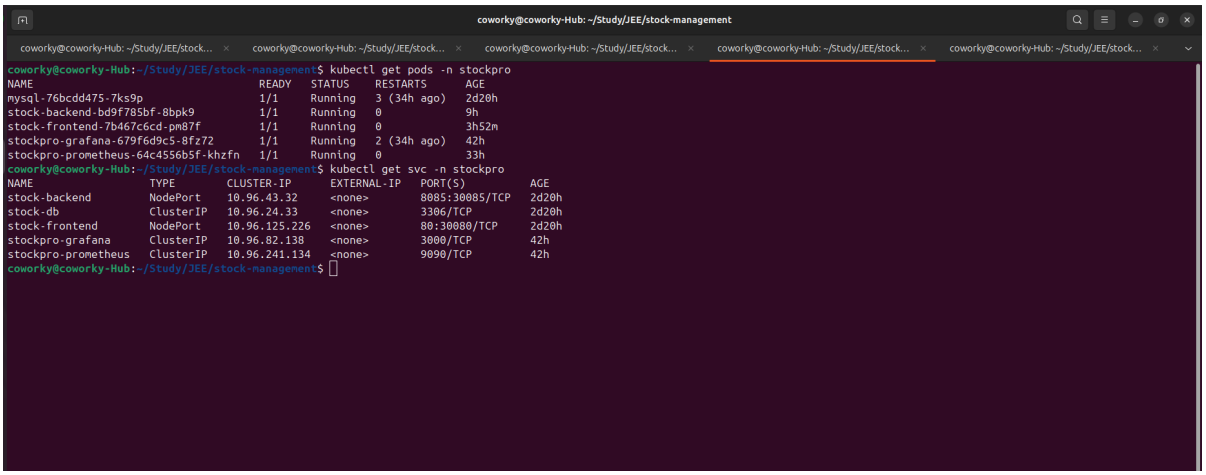


FIGURE 7 – Workflows CI, Security et CD validés sur le même SHA

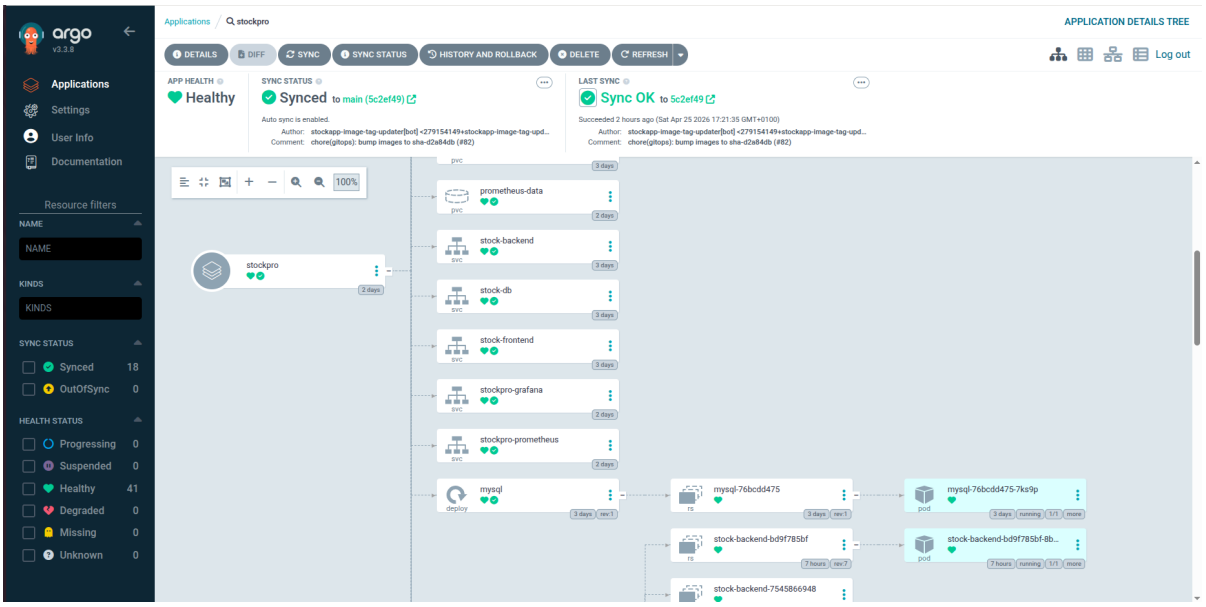


FIGURE 8 – Pull request GitOps de mise à jour des images

5.2. Logique des pipelines

5.2.1 Pipeline CI

Le workflow `ci.yml` constitue le socle de validation continue du projet. Il regroupe la vérification du backend avec `./mvnw verify`, les étapes frontend autour de `npm ci`, des tests, du build et du contrôle de formatage, puis l'analyse SonarCloud. J'ai également prévu une logique particulière pour les **PR GitOps bot**, afin d'éviter le déclenchement inutile de traitements lourds lorsque les changements ne concernent que la mise à jour GitOps.

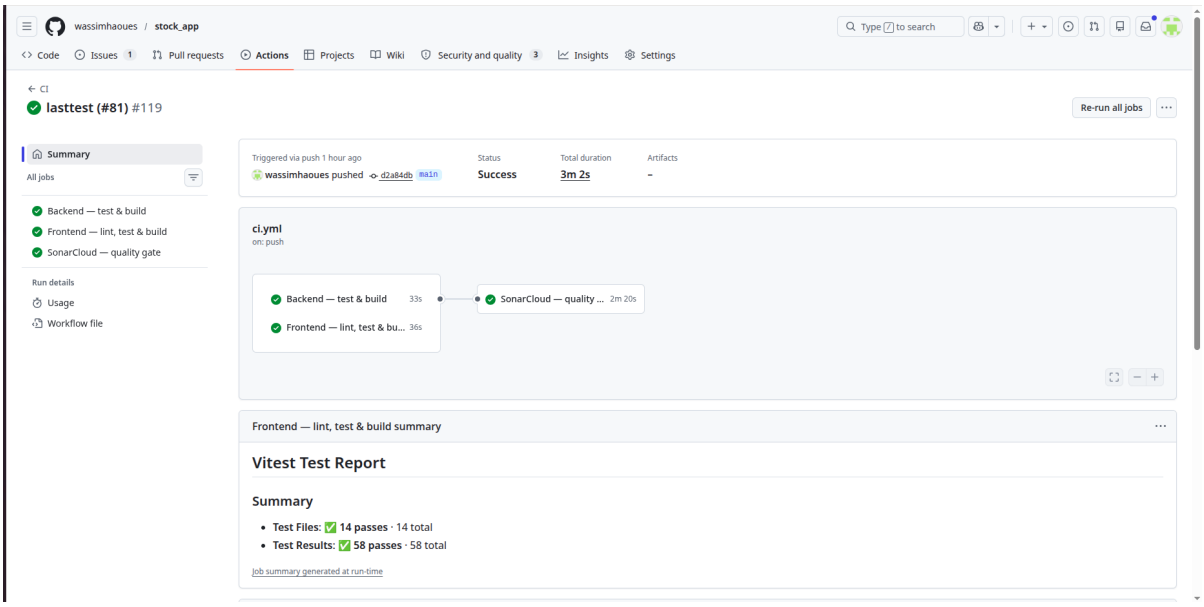


FIGURE 9 – Pipeline CI GitHub Actions du projet

5.2.2 Pipeline Security

Le workflow `security.yml` complète la CI par une couche de sécurité dédiée. Il exécute CodeQL sur les technologies `java-kotlin` et `javascript-typescript`, ajoute OWASP Dependency-Check pour la partie Maven et lance Trivy sur les images Docker. Ce pipeline ne sert donc pas seulement à détecter des erreurs de code, mais aussi à identifier des vulnérabilités potentielles dans les dépendances et les artefacts construits. Il dispose en plus d'un déclenchement hebdomadaire programmé le lundi à `03:00 UTC`.

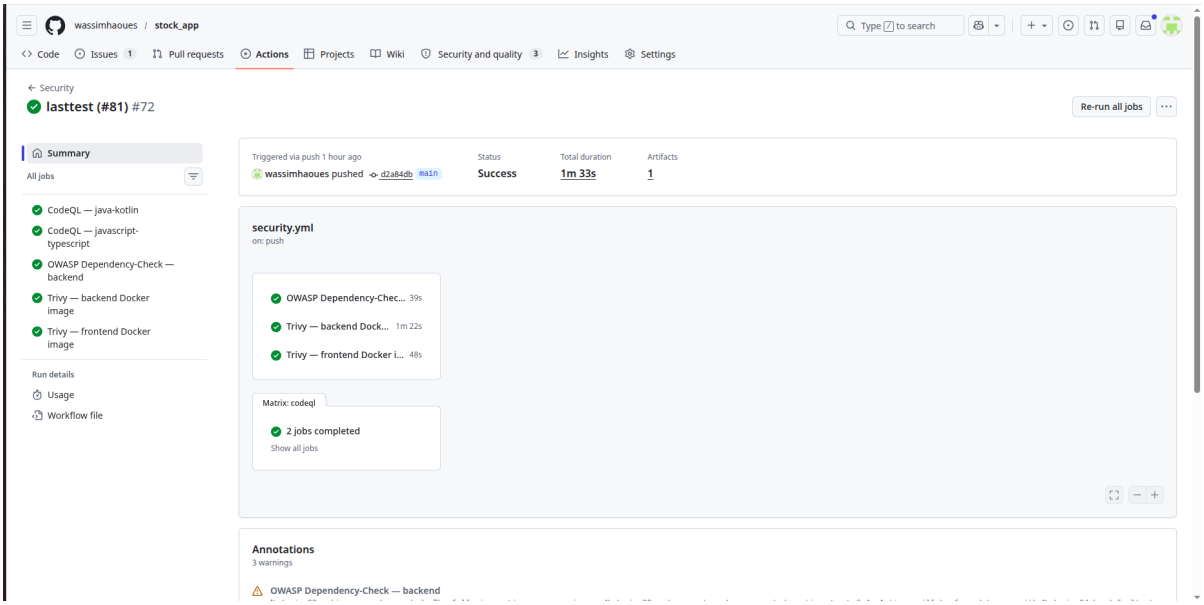


FIGURE 10 – Pipeline Security GitHub Actions du projet

5.2.3 Pipeline CD

Le workflow `cd.yml` ne se limite pas à publier automatiquement après un push sur `main`. Il met en œuvre une logique de livraison contrôlée, qui peut être résumée en plusieurs étapes successives :

1. `wait-for-main-validations` vérifie explicitement que `CI` et `Security` sont verts sur le même SHA ;
2. `detect-changes` calcule un tag immuable `sha-<7 caractères>` et repère les changements dans `backend/` ou `frontend/` ;
3. `build-backend` et `build-frontend` ne reconstruisent que ce qui est nécessaire ;
4. `gitops-bump` crée une branche `gitops/bump-images-sha-*`, met à jour `k8s/overlays/gitops/kustomization.yaml`, ouvre une pull request GitOps puis active l'auto-merge.

Deux scripts Python, `update_gitops_kustomization.py` et `render_gitops_pr_body.py`, contribuent à rendre ce flux plus

stable et plus déterministe.

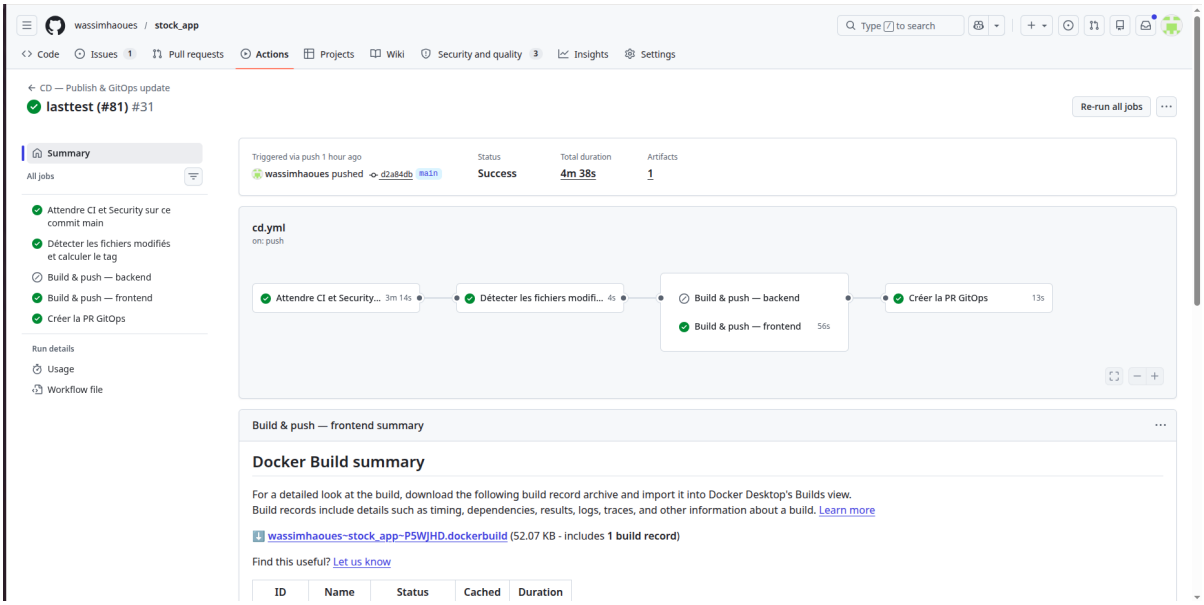


FIGURE 11 – Pipeline CD GitHub Actions du projet

6. Historique Git, stratégie de branches et collaboration

6.1. Stratégie observable

J'ai organisé le travail par phases. Les branches `feature/phase-*` correspondent surtout au socle applicatif, alors que les branches `feature/devops-phase-*` concernent l'industrialisation. On retrouve notamment `main`, `dev`, les branches de `feature/phase-0-planning` à `feature/phase-11-tests-cleanup`, puis celles de `feature/devops-phase-12-local-baseline` à `feature/devops-phase-23-finalization`. J'ai aussi utilisé des branches temporaires de stabilisation comme `testpipeline`, `feature/testglobal` et `lasttest`.

Le flux principal que j'ai suivi est de type `feature -> dev -> main -> GitOps PR`. Les passages directs vers `main` correspondent surtout à des tests et à des ajustements de validation.

6.2. Phases majeures observables

Date	Commit	Ce que j'en retiens
22/04/2026	`efc23db`	mise en place de la CI de base
22/04/2026	`52b6802`	ajout des quality gates et des scans de sécurité
23/04/2026	`7cd25f8`	premiers manifests Kubernetes et exécution locale sur kind
23/04/2026	`fc7b21d` puis `4581737`	introduction d'ArgoCD puis activation de l'auto-sync avec `selfHeal`
23/04/2026	`d9ed468`	premier workflow CD avec build et push d'images
23/04/2026	`1dd49de`	centralisation du logging backend
24/04/2026	`26dfa66`	ajout de l'observabilité avec Prometheus et Grafana
24/04/2026	`4917a35`	blocage du CD tant que `CI` et `Security` ne sont pas validés
24/04/2026	`14c1f70`	remplacement du push GitOps direct par une pull request GitOps
24/04/2026	`033bf26`	validation légère dédiée aux PR GitOps
25/04/2026	`08080bf`, `5af223e`, `4626bb9`	phase finale de stabilisation du flux GitOps bot

Dans la fin de l'historique, j'ai aussi vu plusieurs retours de `main` vers certaines branches de travail. Pour moi, cela montre surtout une phase de correction et de durcissement du pipeline plutôt qu'une évolution fonctionnelle du produit.

7. Problèmes rencontrés

En suivant les commits et les workflows, j'ai pu relever plusieurs problèmes concrets rencontrés pendant la mise en place de la chaîne DevOps :

1. **La gouvernance du pipeline CD devait être renforcée.** La publication d'image ne devait pas pouvoir démarrer tant que les validations essentielles n'étaient pas terminées et cohérentes sur le même état du code.
2. **Le flux GitOps manquait de contrôle au départ.** La mise à jour de la source GitOps était trop directe et ne laissait pas assez de place à la revue, à la traçabilité et aux validations intermédiaires.
3. **Les validations automatiques des PR GitOps étaient instables.** Le comportement des contrôles associés au bot GitOps n'était pas encore suffisamment prévisible pour garantir un flux fluide et fiable.
4. **La gestion des manifests YAML restait fragile.** La génération, le formatage et la validation des manifests pouvaient encore échouer pour des raisons de structure ou d'outillage insuffisamment robuste.
5. **Certaines adaptations entre environnements n'étaient pas encore stabilisées.** Des différences entre l'exécution locale et Kubernetes, notamment autour du CORS et de certains comportements techniques, compliquaient l'intégration de bout en bout.
6. **L'observabilité Kubernetes n'était pas immédiatement fiable.** La collecte des métriques et la supervision du backend ont nécessité des ajustements avant d'offrir une visibilité correcte sur l'état du système.

8. Solutions apportées

Pour répondre à ces problèmes, j'ai mis en place les solutions suivantes :

1. **Renforcement de la gouvernance du CD.** La publication est désormais conditionnée par la réussite préalable des workflows indispensables, ce qui sécurise le passage entre validation et livraison.
2. **Passage à un flux GitOps piloté par pull request.** Les mises à jour GitOps sont intégrées par une PR dédiée, ce qui améliore la revue, la traçabilité et le contrôle des changements.
3. **Stabilisation des validations automatiques.** Les contrôles ont été mieux séparés entre validations lourdes et validations légères afin de rendre les traitements plus fiables et mieux adaptés au contexte.
4. **Fiabilisation de la gestion des manifests.** La mise à jour des fichiers YAML a été rendue plus déterministe, et la validation des manifests a été renforcée avec un outil plus adapté.
5. **Externalisation des paramètres dépendants de l'environnement.** Certaines configurations comme le CORS ont été injectées selon le mode d'exécution pour faciliter le passage entre local et Kubernetes.
6. **Consolidation de l'observabilité.** La supervision a été corrigée et enrichie, à la fois par un meilleur branchement des métriques en Kubernetes et par l'ajout d'indicateurs métier exposés par le backend.

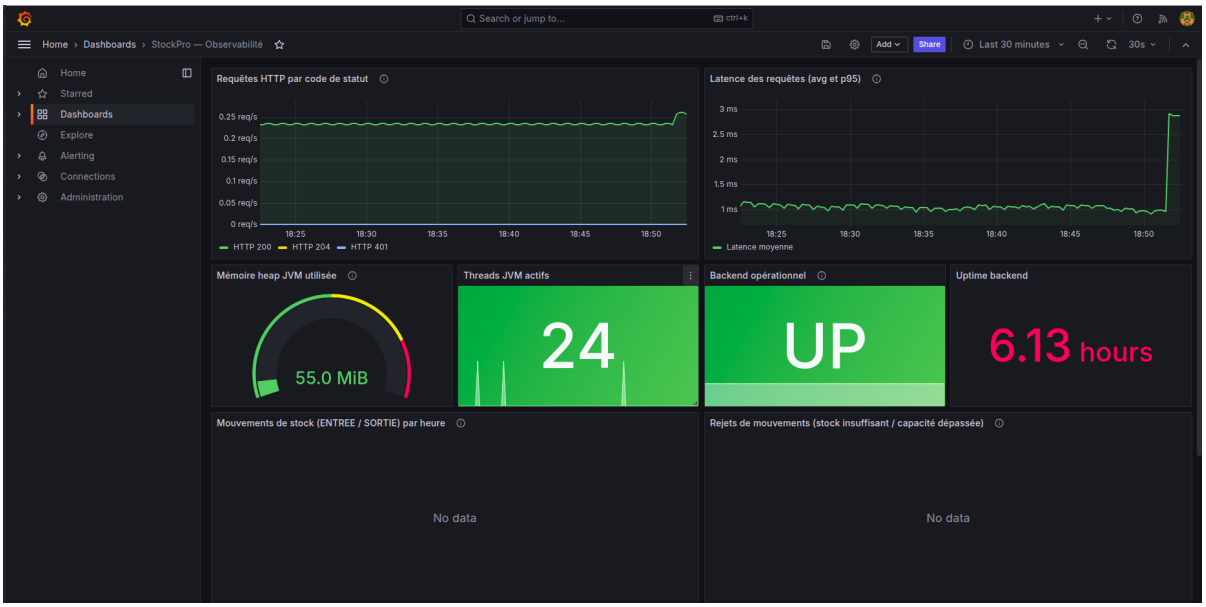


FIGURE 12 – Dashboard Grafana `StockPro - Observabilité`

9. Conclusion

Au final, je considère que mon projet montre bien une implémentation DevOps de bout en bout, cohérente et complète. La partie applicative est importante, mais le point le plus intéressant reste selon moi la chaîne complète qui relie la CI, les scans de sécurité, la publication d'images, le flux GitOps par pull request, le déploiement local sur `kind` et la supervision avec Prometheus et Grafana.

Le principal apport de cette organisation est la traçabilité : un même SHA Git peut être relié à une image GHCR, puis à une pull request GitOps, puis à une révision synchronisée par ArgoCD. Si je devais citer des pistes d'amélioration réalistes, je retiendrais surtout une meilleure gestion des secrets, un mécanisme de rollback plus outillé, un vrai système de migration de schéma et une couche d'alerting plus complète côté monitoring.